

Software Engineering

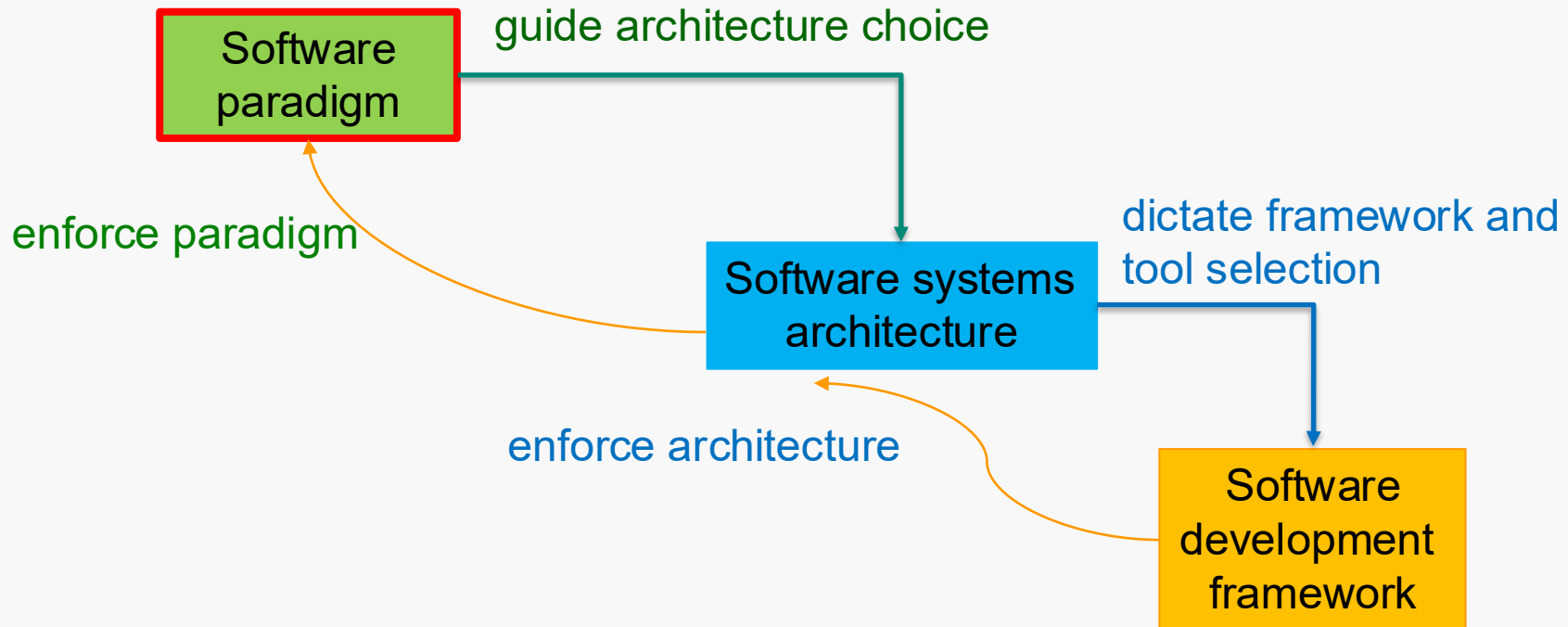
- Software Paradigms

Dr Lily Sun(Dr. Nisha Singh)

Learning Objectives

- ❑ Understand software paradigms in software development
- ❑ Classifications and features of software paradigms
- ❑ AI empowerment to software paradigms

SE Conceptual Flow



overall process to manage and execute software development project

Software development methodology

Software Paradigms

❑ Software paradigms

- ✿ theoretical approaches that guide how software is conceptualised, structured, designed, and implemented
- ✿ design philosophies, tooling, and the mental model developers adopt when solving computational problems
- ✿ ensuring software quality through clear separation of concerns and appropriate abstraction
- ✿ reinforce code structured under clear paradigms is easier to extend and debug, consistent code practices across teams
- ✿ adapting to problem domains, e.g., functional for data transformations, object-oriented for modelling real-world aspects

Classifications of Software Paradigms

Procedural

Object-Oriented

Functional

Logical

Declarative

Event-Driven

Aspect-Oriented

Concurrent/Parallel

Software Paradigms (cont.)

❑ Procedural Programming Paradigm

- ✿ based on the concept of procedure calls (functions)
- ✿ code is organised into procedures or routines
- ✿ programming languages: C, Pascal, Fortran
- ✿ design representations:
 - flowchart of function calls
 - boxes represent procedures, arrows show flow of control
 - hierarchical or linear flow showing sequence, selection, and iteration

Software Paradigms (cont.)

❑ Object-Oriented Programming (OOP) Paradigm

- ✎ organises software design around objects represented by the data and the methods that operate on them
- ✎ programming languages: Java, C++, Python (partially)
- ✎ design representations
 - UML Class Diagrams
 - classes with attributes and methods
 - relationships like inheritance, aggregation, association
 - encapsulation, polymorphism, and inheritance in structure

Software Paradigms (cont.)

❑ Functional Programming Paradigm

- ✎ considers computation as the evaluation of mathematical functions
- ✎ avoids changing-state and mutable data to introduce bugs
- ✎ Programming languages: Haskell, Lisp, Scala (partially)
- ✎ design representations
 - Function composition diagrams (showing data flow between functions)
 - Recursion trees instead of loops

Software Paradigms (cont.)

□ Logical Programming Paradigm

- ✿ based on formal logic
- ✿ in a set of judgements within logical form, expressing facts and rules about some problem domain
- ✿ programming languages: Prolog
- ✿ design representations

● Predicate Logic Tree

presenting facts and rules

Tree-like inference diagrams representing goal resolution (e.g., using backward chaining)

Software Paradigms (cont.)

❑ Declarative Programming Paradigm

- 🌿 focuses on what the program should accomplish rather than detailing how to achieve it
- 🌿 programming languages: SQL, HTML, Prolog
- 🌿 design representations
 - High-level abstraction layer vs execution engine
 - contrast with procedural paradigm using a block that represents desired outcomes instead of steps

Software Paradigms (cont.)

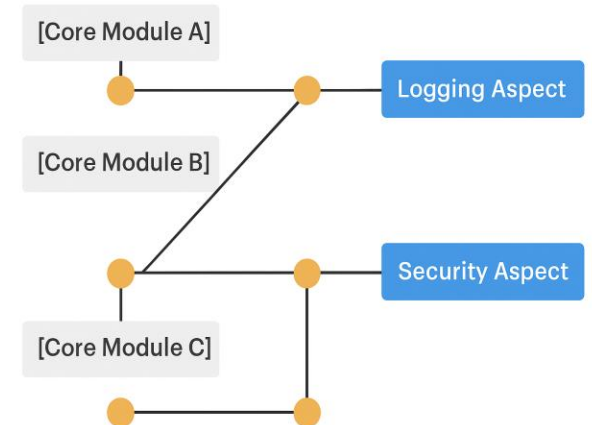
❑ Event-Driven Programming Paradigm

- 🦋 program flow is determined by events, like user actions, sensor outputs, or message passing
- 🦋 programming languages: JavaScript (for web), Visual Basic
- 🦋 design representations
 - Event Handling Flow Diagram
 - events trigger event handlers or callbacks
 - arrows from events to functions with feedback loops

Software Paradigms (cont.)

Aspect-Oriented Programming (AOP) Paradigm

- separates cross-cutting concerns (as distinct modules) that affect multiple parts of the system, but not part of the core business logic
- e.g., logging aspect, security (authentication, authorisation) aspect, error handling aspect, transaction management aspect, performance monitoring aspect
- programming languages: AspectJ (Java), Spring AOP
- design representations
 - Cross-Cutting Concerns Overlay (components + ports)
 - core modules shown with "aspect" modules intertwined with multiple points (join points)



Software Paradigms (cont.)

❑ Concurrent and Parallel Programming Paradigm

- ✎ focus on programs that execute simultaneously on multiple processors or threads
- ✎ used in design for: operating systems, distributed systems, real-time systems
- ✎ programming languages: Java (threads), Erlang
- ✎ design representations
 - Thread/Process Diagram
 - multiple execution threads shown with synchronisation points, shared resources, and message passing

Classifications of Software Paradigms

Paradigm	Focus	Key Concepts	Typical Use Cases
Procedural	Control flow	Procedures, control structures	System programming
Object-Oriented	Objects and classes	Inheritance, encapsulation	Application development
Functional	Functions	Immutability, recursion	Data processing
Logical	Rules and facts	Pattern matching, inference	AI, theorem proving
Declarative	Desired outcomes	Abstract specifications	Queries, configurations
Event-Driven	Events and handlers	Listeners, callbacks	UI programming
Aspect-Oriented Programming	Cross-cutting concerns	Join points, weaving	Logging, security
Concurrent/Parallel	Simultaneous tasks	Threads, synchronization	High-performance computing

Software Paradigms (cont.)

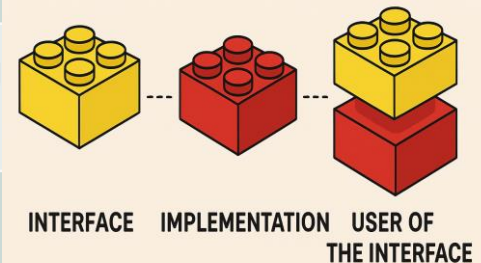
❑ Component-based paradigm

- 🌿 organises software systems as collections of reusable components
- 🌿 a component seen as a modular, encapsulated, and interchangeable part of a system
- 🌿 a component exposed other components via a well-defined interface
- 🌿 applied to inside a system/application

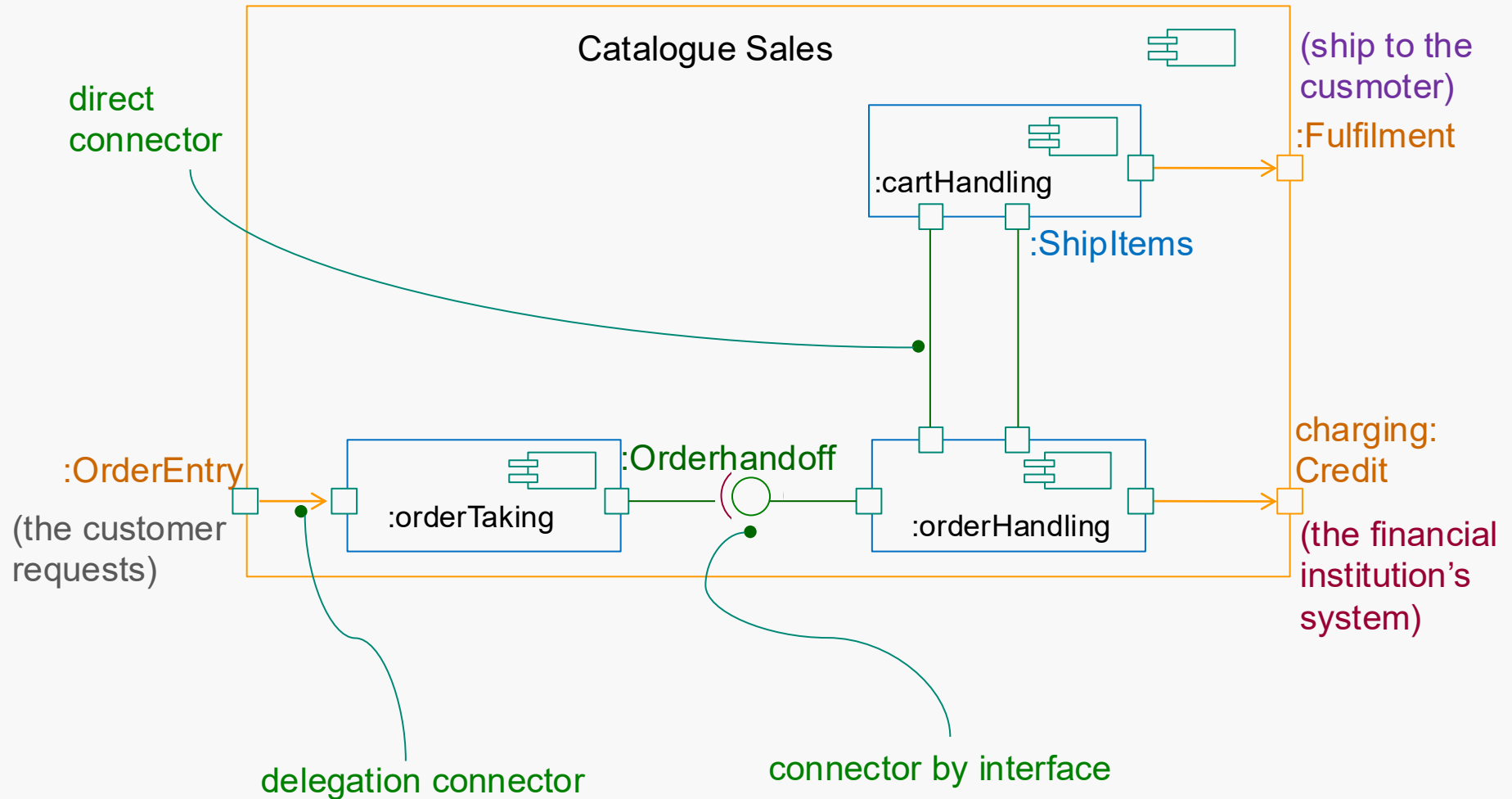
Software Paradigms (cont.)

❑ Component-Based Paradigms

Concept	Description
Component	A self-contained unit that provides specific functionality via interfaces (like a class or module, but independently deployable).
Interface	A specification of services provided/required, hiding the internal implementation.
Encapsulation	Internal implementation details are hidden.
Reusability	Components can be reused across different systems or contexts.
Composition	Systems are built by assembling pre-made components.



Example of Component Diagram



Software Paradigms (cont.)

❑ Service-oriented paradigm

- ✎ focus on building systems as collections of loosely-coupled services
- ✎ a service is a distributed, autonomous unit that performs a specific business task
- ✎ a service communicates with other services over a network via standard protocols (e.g., HTTP, SOAP, REST)
- ✎ applied to between systems (e.g., Microservices, REST APIs in distributed systems)

Software Paradigms (cont.)

❑ Service-oriented paradigm

Concept	Description
Service	A remote-accessible, stateless functionality unit that can be published and consumed over a network.
Loose Coupling	Services are independent and unaware of each other's internal structure.
Interoperability	Services can work across platforms/languages using open standards.
Discovery	Services can be found and bound to dynamically (via registries).
Contracts	Services define a contract (WSDL, API docs) describing their interface.

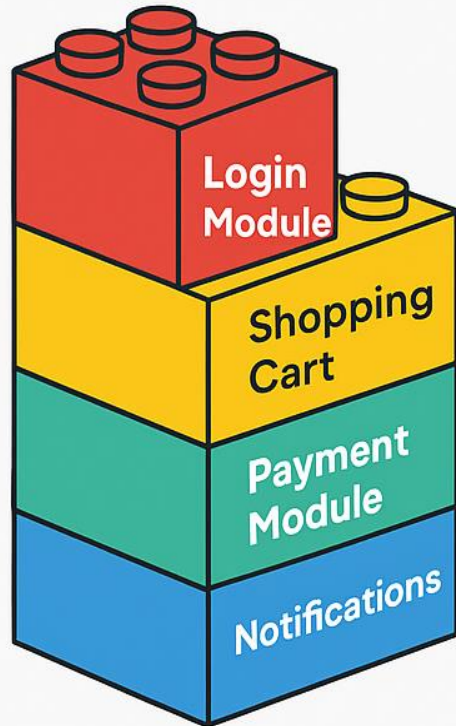
Software Paradigms (cont.)

❑ Component-Based vs Service-Oriented

Aspect	Component-Based	Service-Oriented
Deployment	Local to the application	Network-based, remote
Granularity	Fine-grained	Coarse-grained
Interaction	In-process method calls	Out-process calls (HTTP, SOAP)
Coupling	Tightly or moderately coupled	Loosely coupled
Reuse	Within a system or application	Across applications and organisations
Protocol	Language-specific	Language-agnostic (REST, SOAP)

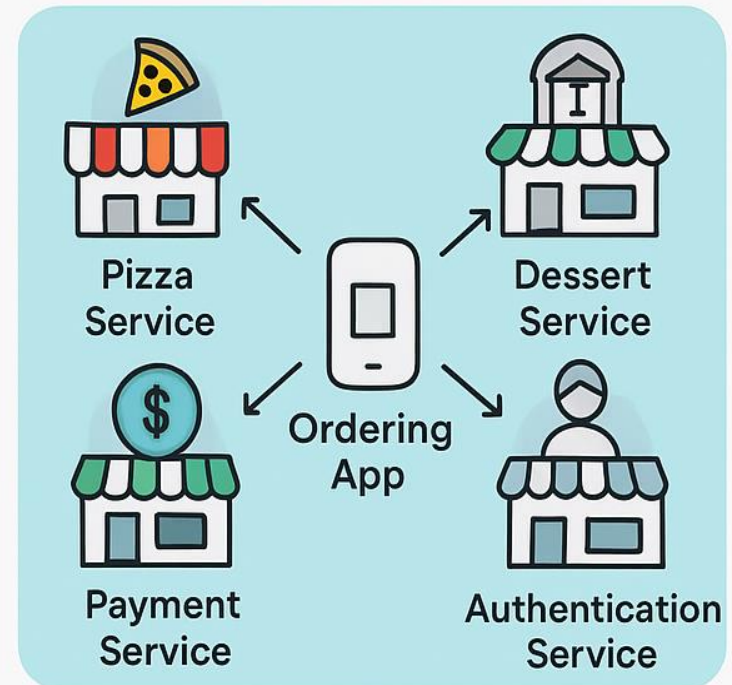
Component-Based vs Service-Oriented Paradigm

Component-Based



All components fit together directly inside the same application.
Tightly coupled. Local integration.

Service-Oriented



Each service is autonomous and distributed.
Integration happens via APIs, not direct 'snapping.' Scalable and loosely coupled.



AI-Empowered Traditional Software Paradigms

Procedural

Dynamic rule generation via ML



Object-Oriented

Objects gain intelligence, self-configuring



Event-Driven

Events predicted/generated by AI



Functional

ML models as pure functions/pipelines



Declarative

AI optimizes queries/U/layouts

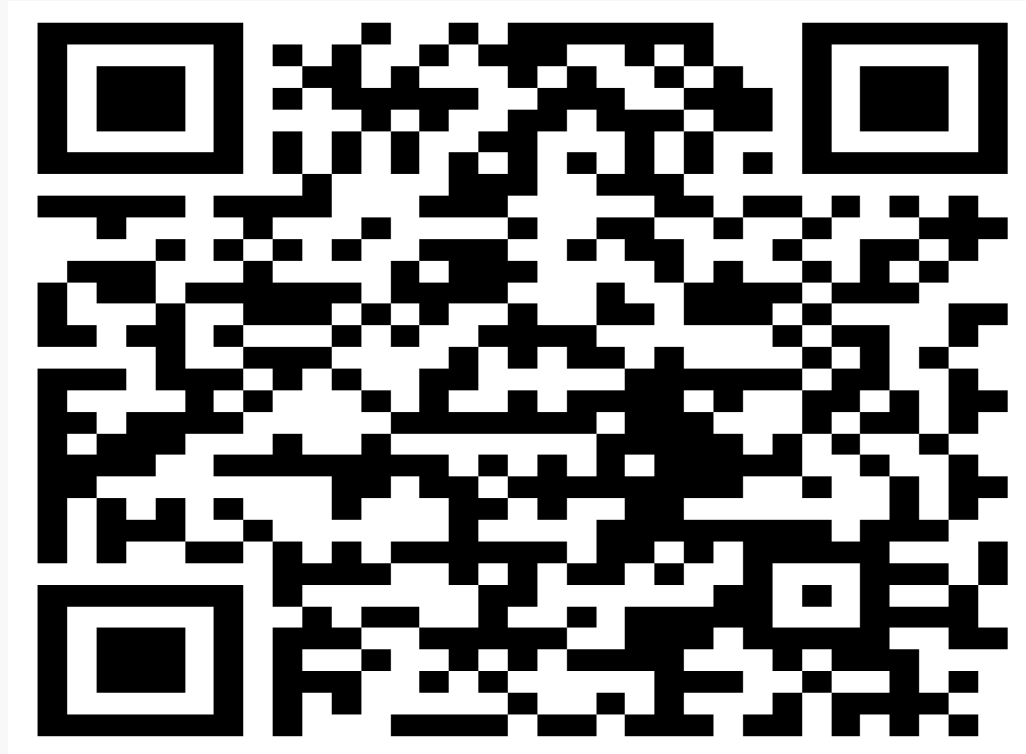


✨ AI doesn't replace paradigms—it **empowers and amplifies** them with **prediction, intelligence, and adaptability**.

Software Paradigm Empowered by AI

Paradigm	Type Description
Cognitive Systems	Software that mimics human reasoning (ML integrated) to adapt behaviour over time, interpret decisions
ML-Driven Architecture	Software designed around ML lifecycle and inference, feedback loops for model retraining
Agent-Based Systems	Autonomous, goal-driven components (AI-based), real-time learning and planning
Conversational UI	AI-based natural language interface (chatbot, voice assistants), context awareness
Data-Centric Design	Designs around data engineering: data pipeline, training flows, data quality, governance, lifecycle, and model inferences

Quiz



<https://forms.office.com/e/VfHzEQcDRt?origin=IprLink>

CS2SE - Software Engineering and Professional Development - Semester 1

